OpenClaw Skills

Crash Course

What Are OpenClaw Skills?

A skill is a folder with documentation and scripts the agent reads before performing a task.

Each skill folder contains:

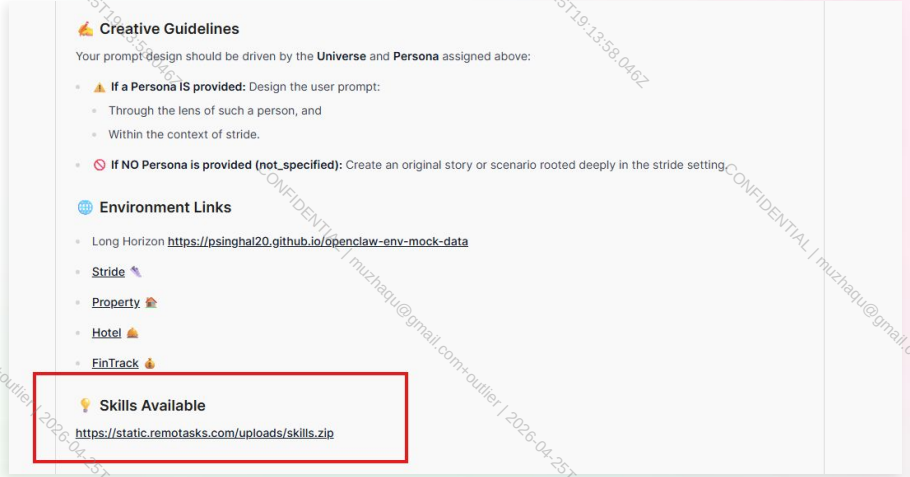
- What data it can access (orders, customers, reservations...)
- What tools and commands are available
- How to use them correctly (parameters, filters, examples)

Without skills, the agent guesses. With skills, it knows exactly what to do.

Where to Find Skills

```
~/.openclaw/skills/
```

- Skills folder is included in every OpenClaw environment.
- Same 30 skills available across all universes, but only some are relevant to each task.
- In each task, you can download a zip file of the skills folder.



✍️ **Creative Guidelines**

Your prompt design should be driven by the **Universe** and **Persona** assigned above:

- ⚠️ **If a Persona IS provided:** Design the user prompt:
  - Through the lens of such a person, and
  - Within the context of stride.
- 🚫 **If NO Persona is provided (not_specified):** Create an original story or scenario rooted deeply in the stride setting.

🌐 **Environment Links**

- Long Horizon https://psinghal20.github.io/openclaw-env-mock-data
- Stride
- Property
- Hotel
- FinTrack

💡 **Skills Available**

https://static.remotasks.com/uploads/skills.zip

What's Inside a Skill Folder

Required:

`SKILL.md`

The most important file. Tells the agent how to perform a task, what tools exits, what commands to run.

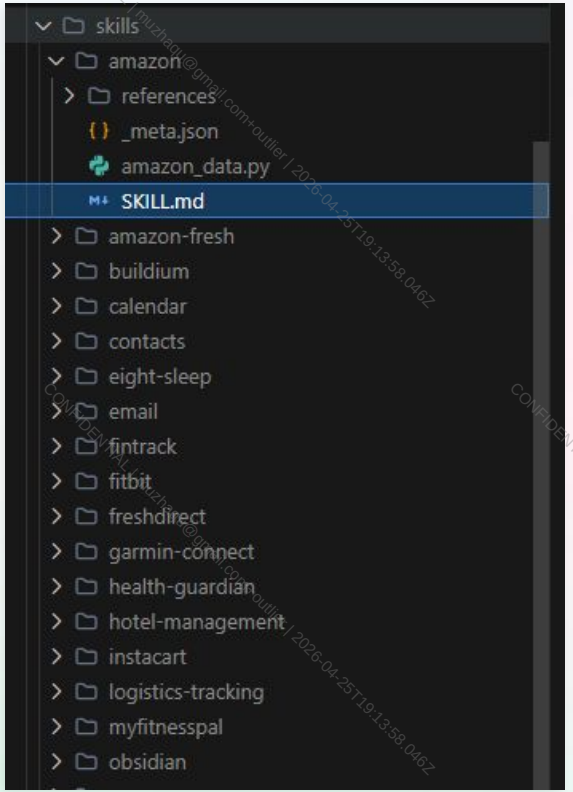
`_meta.json`

Metadata: skill name, description, version. Quick identifier.

Optional:

data_script.py — The script the agent runs with exec tool

references/ — Additional docs or examples for specific scenarios



How to Read SKILL.md

This is the most important file. It tells the agent:

- What data it can access (customers, orders, products, reservations)
- What commands/tools are available (CLI scripts, Python tools)
- What parameters each command accepts (filters, date ranges)
- Example usage patterns, reference to python scripts or workflow.md in reference

Example from shopify/SKILL.md:

```
python3 shopify_data.py search-orders --status returned
python3 shopify_data.py products
python3 shopify_data.py customer <id>
```

Tip: Pay attention to --date-from, --date-to — often key to getting complete data.

Server Skills (Common Skills)

Raw, standalone skills that provide access to individual tools and data sources.

`shopify`

E-commerce data — customers, orders, products

`email`

Read and manage email messages

`contacts`

Contact list and address book

`calendar`

Events, scheduling, availability

`zendesk`

Support tickets and customer service

`zillow`

Real estate listings and property data

`stripe`

Payment processing and transaction data

`obsidian`

Notes and knowledge base

These are shared across all universes. The agent uses them as raw tools.

Workflow Skills (Universe Skills)

Combined skills tailored to each environment. They define how the agent should work in that universe.

Stride Shoes	→	`stride-support/SKILL.md`
Hotel Manager	→	`hotel-management/SKILL.md`
Property Mgmt	→	`property-manager/SKILL.md`
FinTrack	→	`fintrack/SKILL.md`

The first step for most of golden trajectories: read the universe's workflow skill.

How the Agent Uses Skills in Trajectories

Prompt: Analyze all returned orders for Stride Shoes. Identify the most frequent return reason.

1 **read** `~/.openclaw/skills/stride-support/SKILL.md (Reading Universe skill)`

Reasoning: *"Let me see what tools are available for Stride Shoes."*

Result: Learns that Shopify has order data, sees reference to shopify skill

2 **read** `~/.openclaw/skills/shopify/SKILL.md (Reading Server skill)`

Reasoning: *"The stride-support skill mentions Shopify. Let me learn the query commands."*

Result: Learns search-orders command and its parameters (--status, --date-from, --date-to)

3 **exec** `python3 shopify_data.py search-orders --status returned`

Reasoning: *"Now I know the exact command from the SKILL.md. Let me query all returned orders."*

Result: Gets JSON with order data including return reasons

read = open a file **exec** = run a command

Why This Matters for Your Task

When writing a prompt:

- Your prompt must be solvable using available skills
- Agent needs to DISCOVER which skill to use
- Understanding skills helps write accurate hints and outcomes

When reviewing trajectories:

- Check if model found and used the right skills
- Check if it read SKILL.md for correct commands
- Check if it used the right parameters

Skill Gap / Self-Extension tasks:

- If the task requires creating a new skill, golden trajectory should produce one following the same structure: SKILL.md, _meta.json (requried); data script, references/ (optional)

Quick Reference Checklist

Before starting any task:

1. Download the skills/ folder for your task
2. Find the skill related to your universe
   (stride-support, hotel-management, property-manager, or fintrack)
3. Read the SKILL.md to understand the tools and data sources
4. Check for references/workflows.md for domain-specific procedures
5. Check for any data scripts (.py files); these are the scripts the agent can run

Video Walkthrough